

Experiment Concept: Function categories

Objective:

To enhance students learning with the ability to analyze, design, and implement modular programs using arrays and structures. The lab emphasizes function decomposition, data encapsulation, and structured problem-solving.

Definition of a Function: A function is a self-contained block of code designed to perform a specific task. It enhances modularity, promotes code reuse, and simplifies debugging by allowing programmers to divide complex problems into manageable subroutines.

Function classification:

- ✓ Without Arguments and Without Return Value
- ✓ Without Arguments but With Return Value
- ✓ With Arguments but Without Return Value
- ✓ With Arguments and With Return Value

Function with no return and no parameters: A function that performs a task without taking input or returning a value.

Function neither takes input nor returns output. It uses variables declared inside the function or globally.

Example:

```
#include <stdio.h>
void showMessage()
{
    int x = 10; // Local variable
    printf("Value of x is: %d\n", x);
}

int main()
{
    showMessage();
    return 0;
}
```

Function with no return but with parameters: A function that accepts input but does not return any value.

Function takes input but does not return anything. It uses the input variable to perform operations.

Example:

```
#include <stdio.h>
void printSquare(int n)
{
    int square = n * n; // Local variable
    printf("Square of %d is: %d\n", n, square);
}

int main()
{
    printSquare(5);
    return 0;
}
```

Function with return and no parameters: A function that returns a value but does not require any input.

Function does not take input but returns a value using internal variables.

Example:

```
#include <stdio.h>

int getNumber()
{
    int num = 42; // Local variable
    return num;
}

int main()
{
    int result = getNumber();
    printf("Returned number is: %d\n", result);
    return 0;
}
```

Function with return and parameters: A function that takes input and returns a computed result.

Function takes input and returns a result using variables.

Example:

```
#include <stdio.h>

int addNumbers(int a, int b) {
    int sum = a + b; // Local variable
    return sum;
}

int main() {
    int result = addNumbers(10, 20);
    printf("Sum is: %d\n", result);
    return 0;
}
```

Benefits of Functions in Programming

- ✓ **Modularity:** Breaks complex problems into manageable sub-tasks and promotes logical organization and clarity in code
- ✓ **Reusability:** Functions can be reused across multiple programs or modules and reduces redundancy and improves development efficiency
- ✓ **Maintainability:** Easier to debug, test, and update individual components and changes in one function don't affect others if properly encapsulated
- ✓ **Abstraction:** Hides internal implementation details and allows programmers to focus on what a function does, not how
- ✓ **Scalability:** Facilitates expansion of programs by adding new functions and supports team-based development with clear function boundaries
- ✓ **Improved Readability:** Named functions make code self-explanatory and enhances documentation and peer understanding
- ✓ **Encapsulation:** Keeps data and operations tightly bound, promotes secure and predictable behavior
- ✓ **Testing and Debugging:** Functions can be unit-tested independently and isolates errors and simplifies troubleshooting
- ✓ **Supports Recursion**

FUNCTION CATEGORIES IN C

A function is a block of code that performs a specific task. Functions help in modularity, reusability and readability.

1. BASED ON ORIGIN

2. BASED ON PARAMETERS AND RETURN VALUE

A. LIBRARY FUNCTIONS (Predefined Functions)

- Built-in functions in C library
- Already defined
- We just call and use them

Examples:
printf(), scanf(),
sqrt(), strlen()

Example:
#include <stdio.h>

```
int main() {  
    printf("Hello World");  
    return 0;  
}
```

B. USER-DEFINED FUNCTIONS (Programmer Defined)

- Created by the user
- Written to perform specific tasks
- Can be called multiple times

Example:

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int sum = add(5, 3);  
    printf("Sum = %d", sum);  
    return 0;  
}
```

A. NO ARGUMENTS NO RETURN VALUE

- No input
- No output
- Performs task only

Example:

```
void greet() {  
    printf("Hello\n");  
}  
  
int main() {  
    greet();  
    return 0;  
}
```



Used when we only need to perform some action.

B. ARGUMENTS NO RETURN VALUE

- Takes input
- No output
- Processes the input

Example:

```
void printSquare(int x) {  
    printf("Square = %d",  
        x * x);  
}  
  
int main() {  
    int n = 5;  
    printSquare(n);  
    return 0;  
}
```



Used when we pass data to function for processing.

C. NO ARGUMENTS WITH RETURN VALUE

- No input
- Returns output
- Gives result to caller

Example:

```
int getNumber() {  
    return 100;  
}  
  
int main() {  
    int num = getNumber();  
    printf("Number = %d",  
        num);  
    return 0;  
}
```



Used when function calculates and returns a value.

D. ARGUMENTS WITH RETURN VALUE

- Takes input
- Returns output
- Most commonly used

Example:

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int sum = add(4, 6);  
    printf("Sum = %d",  
        sum);  
    return 0;  
}
```



Used when we pass data and get result back.



SUMMARY

Functions can be classified by where they come from (library or user-defined) and by how they use data (parameters and return values). Choosing the right type of function helps in writing efficient and reusable programs.

Concepts to understand for passing and returning character, number, string, array, sentence

PASSING AND RETURNING DIFFERENT DATA TYPES IN C

1. CHARACTER

Program:

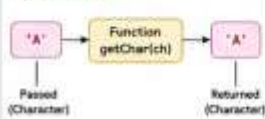
```
#include <stdio.h>
char getChar(char ch)
{
    return ch;
}

int main()
{
    char c = 'A';
    printf("Character = %c",
        getChar(c));
    return 0;
}
```

Output:

Character = A

How it works:



2. NUMBER (INTEGER)

Program:

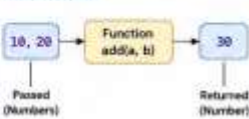
```
#include <stdio.h>
int add(int a, int b)
{
    return a + b;
}

int main()
{
    int result;
    result = add(10, 20);
    printf("Sum = %d", result);
    return 0;
}
```

Output:

Sum = 30

How it works:



3. STRING

Program:

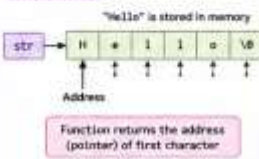
```
#include <stdio.h>
char* getString()
{
    return "Hello";
}

int main()
{
    char *str;
    str = getString();
    printf("String = %s", str);
    return 0;
}
```

Output:

String = Hello

How it works:



4. ARRAY

Program:

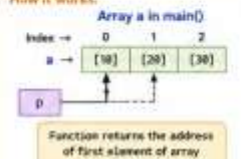
```
#include <stdio.h>
int* getArray(int a[])
{
    a[0] = 10;
    a[1] = 20;
    a[2] = 30;
    return a;
}

int main()
{
    int a[3];
    int *p;
    p = getArray(a);
    printf("%d %d %d",
        p[0], p[1], p[2]);
    return 0;
}
```

Output:

10 20 30

How it works:



5. SENTENCE (STRING)

Program:

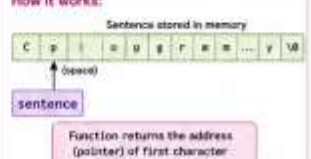
```
#include <stdio.h>
#include <string.h>
char* getSentence(char str[])
{
    return str;
}

int main()
{
    char sentence[100] =
        "C programming is easy";
    printf("Sentence = %s",
        getSentence(sentence));
    return 0;
}
```

Output:

Sentence = C programming is easy

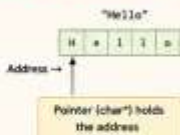
How it works:



WHY POINTERS FOR STRING AND ARRAY?

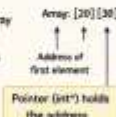
Strings:

A string is a character array ended with '\0'. Functions cannot return whole array directly. So, the address of first character is returned using char*.



Arrays:

An array name represents the address of its first element. Functions cannot return whole array directly. So, the address of first element is returned using int*.



SUMMARY

Data Type	Passing in Function	Returning from Function	Example
Character	char	char	getChar(ch)
Number	int / float / etc.	int / float / etc.	add(a, b)
String	char[] or char *	char *	getString()
Array	int[] (decays to int*)	int *	getArray(a)
Sentence	char[]	char *	getSentence(str)



KEY POINT TO REMEMBER: Single value → return the value.

Collection of values (array/string) → return its address using pointer.

In C, strings, sentences, and arrays are generally handled using pointers/addresses when passed to or returned from functions.

Easy way to remember

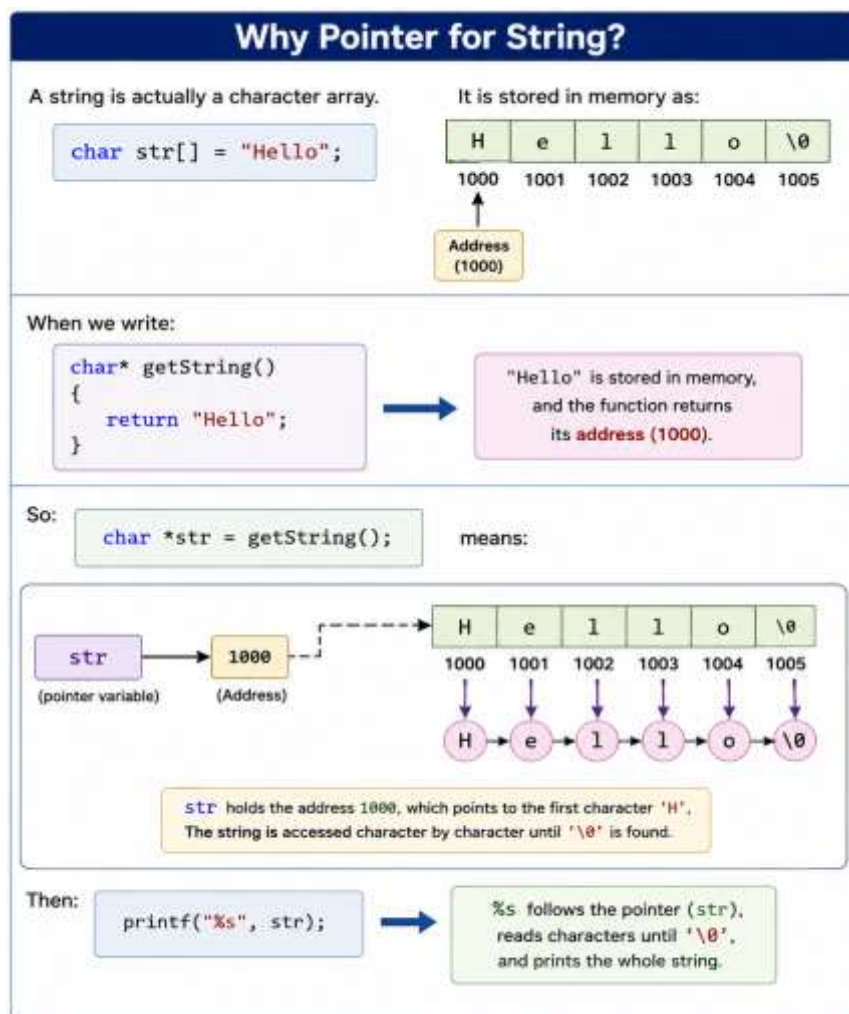
Data	Passing	Returning
Character	char	char
Number	int	int
String	char[] / char *	char *
Array	int[]	int *
Sentence	char[]	char *

In C, arrays and strings cannot be directly returned from a function. A pointer is used to return the address of the first element.

Data	Can function return directly?	Example
int	Yes	return 10;
char	Yes	return 'A';
Array	Not directly	Return pointer
String	Not directly	Return pointer

Rule to remember:

- Single value → return the value
- Collection of values (array/string) → return its address using a pointer.



Why Pointer for Array?

Consider:

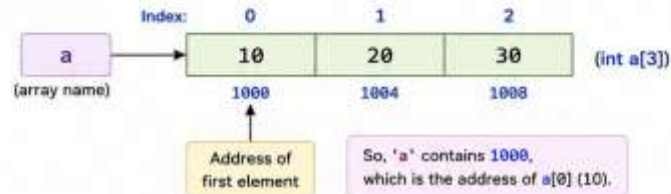
```
int* getArray(int a[])  
{  
    return a;  
}
```

- The function returns a pointer (address) to the first element of the array.
- In C, an array name represents the address of its first element.

When an array is passed:

```
getArray(a);
```

the array name **a** represents the **address** of its first element.



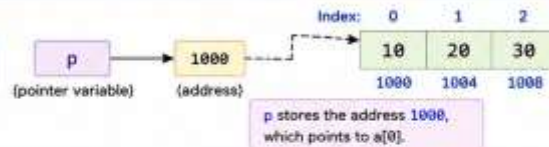
The function returns that address:

```
return a;
```

It returns the address 1000.

So we receive it using:

```
int *p = getArray(a);
```



and access:

```
p[0] → 10  
p[1] → 20  
p[2] → 30
```

Expression	What it means	Value
p[0]	*(p + 0) → value at address 1000	10
p[1]	*(p + 1) → value at address 1004	20
p[2]	*(p + 2) → value at address 1008	30



Summary: Arrays cannot be returned directly in C.
So we return the **address of the first element** using pointer.

Functions with structures

1. Passing and Returning a Structure

```
#include <stdio.h>
struct Student
{
    int roll;
    float marks;
};
struct Student display(struct Student s)
{
    printf("Roll = %d\n", s.roll);
    printf("Marks = %.2f\n", s.marks);
    return s;
}
int main()
{
    struct Student s1, s2;
    s1.roll = 101;
    s1.marks = 85.5;
    s2 = display(s1);
    return 0; }
```

Output:

```
Roll = 101
Marks = 85.50
```

2. Passing and Returning an Array of Structures

```
#include <stdio.h>
struct Student
{
    int roll;
    float marks;
};

struct Student* display(struct Student s[])
{
    int i;
    for(i = 0; i < 3; i++)
    {
        printf("Roll = %d, Marks = %.2f\n",
            s[i].roll, s[i].marks);
    }
    return s;
}

int main()
{
    struct Student s[3];
    struct Student *p;

    s[0].roll = 101;
    s[0].marks = 80.5;

    s[1].roll = 102;
    s[1].marks = 85.5;
```

```
s[2].roll = 103;  
s[2].marks = 90.5;  
  
p = display(s);  
  
return 0;  
}
```

Output:

Roll = 101, Marks = 80.50
Roll = 102, Marks = 85.50
Roll = 103, Marks = 90.50

Rule to remember

Data	Function Return
Single int	int
Single char	char
Single structure	struct Student
Array	int *
Array of structures	struct Student *

1. Passing and Returning a Structure Using Pointer

```
#include <stdio.h>
struct Student
{
    int roll;
    float marks;
};

struct Student* display(struct Student *s)
{
    printf("Roll = %d\n", s->roll);
    printf("Marks = %.2f\n", s->marks);

    return s;
}

int main()
{
    struct Student s1;
    struct Student *p;

    s1.roll = 101;
    s1.marks = 85.5;

    p = display(&s1);

    printf("\nReturned Roll = %d", p->roll);
    printf("\nReturned Marks = %.2f", p->marks);

    return 0;
}
```

Output:

```
Roll = 101
Marks = 85.50
Returned Roll = 101
Returned Marks = 85.50
```

2. Passing and Returning an Array of Structures Using Pointer

```
#include <stdio.h>

struct Student
{
    int roll;
    float marks;
};

struct Student* display(struct Student *s)
{
    int i;

    for(i = 0; i < 3; i++)
    {
        printf("Roll = %d, Marks = %.2f\n",
```

```

        (s + i)->roll,
        (s + i)->marks);
    }

    return s;
}

int main()
{
    struct Student s[3];
    struct Student *p;

    s[0].roll = 101;
    s[0].marks = 80.5;

    s[1].roll = 102;
    s[1].marks = 85.5;

    s[2].roll = 103;
    s[2].marks = 90.5;

    p = display(s);

    printf("\nFirst Student:");
    printf("\nRoll = %d", p->roll);
    printf("\nMarks = %.2f", p->marks);

    return 0;
}

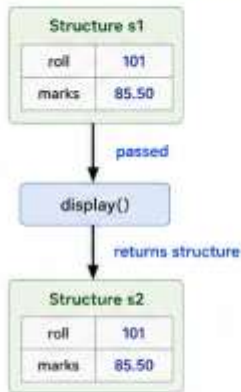
```

Output:

Roll = 101, Marks = 80.50
 Roll = 102, Marks = 85.50
 Roll = 103, Marks = 90.50

First Student:
 Roll = 101
 Marks = 80.50

1 PASSING AND RETURNING STRUCTURE (BY VALUE)



Example:

```
struct Student display(struct Student s)
{
    printf("Roll = %d, Marks = %.2f\n",
        s.roll, s.marks);
    return s; // returns entire structure
}
```

In main():

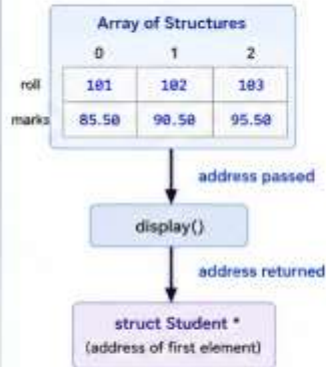
```
struct Student s1, s2;
s2 = display(s1); // copy returned
```



Key Point

Entire structure is passed to function and a copy is returned.

2 PASSING AND RETURNING ARRAY OF STRUCTURES (WITHOUT POINTERS)



Example:

```
struct Student* display(struct Student s[])
{
    // s is address of first element.
    return s; // returns same address
}
```

In main():

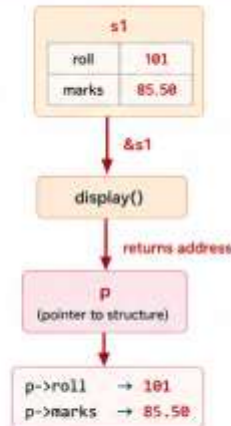
```
struct Student s[3];
struct Student *p;
p = display(s); // p gets address of s[0]
```



Key Point

Array name (s) is address of first element. Function returns the same address.

3 PASSING AND RETURNING STRUCTURE WITH POINTERS



Example:

```
struct Student* display(struct Student *s)
{
    printf("Roll=%d Marks=%.2f\n",
        s->roll, s->marks);
    return s; // returns address
}
```

In main():

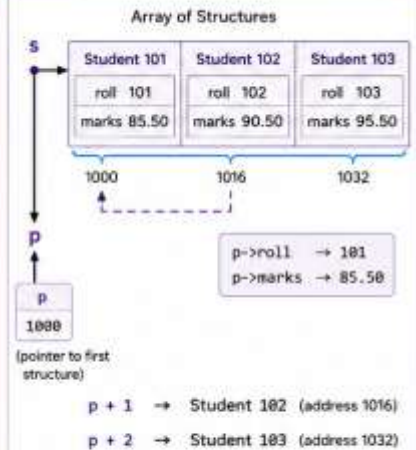
```
struct Student s1, *p;
p = display(&s1); // pass address
printf("%d %.2f", p->roll, p->marks);
```



Key Point

Address of structure is passed. Function returns the same address. Access members using ->

4 HOW THE POINTER WORKS (ARRAY OF STRUCTURES)



Key Point

For an array of structures, we pass the address of the first structure:

```
display(s);
```

Inside the function:

```
struct Student *s
```

Access each structure using:

```
{s + 1} -> roll  
{s + 1} -> marks
```

★ Remember

Structure pointer → ->
Array of structures + pointer → {p + 1}->member

STUDENTS TO ADDRESS FOLLOWING CONCEPTS

3.1 Simple problem-solving using categories of functions

3.1.1. Function Without Arguments and Without Return Value

A travel agency wants to calculate and display the total cost of a trip. The distance (in km), cost per km, and number of travellers are fixed for this trip and declared globally. Write a function that does not take any arguments and does not return any value, but performs the calculation and displays the result directly.

Requirements:

- Declare the following variables globally:
float distance; float costPerKm; int travellers;
- **Function name:** calculateTripCost()
- No arguments, no return value
- Formula: $\text{totalCost} = \text{distance} \times \text{costPerKm} \times \text{travelers}$

Expected output:

Enter values of distance, costperkm, travellers count

distance = 120.5;

costPerKm = 15.0;

Travellers count = 4;

Total cost of the trip: ₹7230.00

3.1.2. Function Type: Fixed-Rate Computation Without Arguments, With Return Value

An electricity board wants to estimate the monthly bill for a household using fixed values for units consumed and rate per unit. These values are declared globally. Write a function that does not take any arguments, but returns the total bill amount.

Requirements:

- Declare the following global variables:
- int unitsConsumed = 250; float ratePerUnit = 5.75; Function name: calculateBill()
- No arguments and should returns float total bill
- Formula:
 $\text{totalBill} = \text{unitsConsumed} \times \text{ratePerUnit}$

Expected output:

Enter values of units and rateperunit

unitsConsumed = 350;

float ratePerUnit = 6.75;

Estimated Monthly Electricity Bill: ₹2362.50

3.1.3. With Arguments but Without Return Value

Design a grading system that prints the grade based on a student's score. The function should accept the score as an argument and print the corresponding grade:

- ≥ 90 : A
- 80–89: B
- 70–79: C
- < 50 : F

Requirements:

- Function name: printGrade(int score).
- Accepts score as argument.
- No return value.
- Use if-else or switch-case.

Expected output:

Enter student's score: 94

Grade: A

Enter student's score: 82

Grade: B

Enter student's score: 75

Grade: C

Enter student's score: 45

Grade: F

3.1.4. With Arguments and With Return Value

A bank wants to calculate the interest earned on a fixed deposit. Write a function that takes principal amount, rate of interest, and time (in years) as arguments and returns the interest earned.

Requirements:

- Function name: calculateInterest (float principal, float rate, int time).
- Returns float value.
- Use formula: $\text{Interest} = (\text{principal} \times \text{rate} \times \text{time}) / 100$.

Expected output:

Enter principal amount: ₹1500000

Enter rate of interest (% per annum): 3

Enter time (in years): 2

Interest earned on fixed deposit: ₹90000.00

3.2. Problem-solving using categories of functions Array and structure Based Function development scenarios

3.2.1. With Arguments but Without Return Value Scenario

A school wants to display the marks of 5 students in a subject. Write a function that accepts an array of marks and prints each mark with its index.

Requirements:

- Function name: printMarks(int marks[], int size)
- Accepts array and its size as arguments
- No return value

Expected output:

Student 1: Aarav - Marks: 78
Student 2: Diya - Marks: 85
Student 3: Karthik - Marks: 92
Student 4: Meera - Marks: 87
Student 5: Rohan - Marks: 88

3.2.2. With Arguments and With Return Value scenario

A retail store wants to calculate the total bill from a list of item prices. Write a function that accepts an array of prices and returns the total cost.

Requirements:

- Function name: calculateTotal(float prices[], int count)
- Accepts array and count as arguments
- Returns float total
- Use loop to sum all prices

Expected output:

1 item price420.5
2 item price56.50
3 item price856.35
4 item price236.5
Total Bill: ₹1569.85

3.2.3. With Arguments but Without Return Value (Structure)

A university wants to print the details of multiple students in a formatted report. Each student has a name, roll number, branch, and CGPA. Write a function that accepts an array of Student structures and the number of students, and prints each student's details in a formatted manner.

Requirements:

- Function name: printStudentDetails(struct Student s[], int count)
- **Consider structure**
struct Student
{
char name[50];
int roll;
char branch[30];
float cgpa;
};
- Accepts: Array of Student structures and Integer count for number of students

- No return value

Expected output:

Student 1:
 Name : Anjali
 Roll No: 101
 Branch : CSE
 CGPA : 8.75

Student 2:
 Name : Ravi
 Roll No: 102
 Branch : ECE
 CGPA : 8.40

Student 3:
 Name : Meera
 Roll No: 103
 Branch : ME
 CGPA : 9.10

3.2.4. With Arguments and With Return Value (Structure)

An HR system wants to update the salaries of multiple employees based on their performance ratings. Each employee has a name, ID, current salary, and rating. Write a function that accepts an Employee structure, applies the salary increment logic, and returns the updated structure. Use an array of employees in main() and update each one using the function.

Requirements

- ✓ Consider structure


```
struct Employee
{
    char name[50];
    int id;
    float salary;
    int rating; // 1 to 5
};
```
- ✓ .Function name: updateSalary(struct Employee e)
- ✓ Accepts one Employee structure
- ✓ Returns updated Employee structure
- ✓ Salary increment logic:
 - Rating 5 → +20%
 - Rating 4 → +10%
 - Rating ≤3 → no change
- ✓ Use an array of employees in main() and update each using a loop

Output expected:

Employee 1:
 Name : Rahul
 ID : 1001
 Initial Salary : ₹50000.00
 Updated Salary : ₹60000.00
 Rating : 5

Employee 2:

Name : Sneha
ID : 1002
Initial Salary : ₹45000.00
Updated Salary : ₹49500.00
Rating : 4

Employee 3:

Name : Arjun
ID : 1003
Initial Salary : ₹40000.00
Updated Salary : ₹40000.00
Rating : 3